Backend Engineering Assignment: “Instant Strike” Execution Engine

Stockwiz Technologies — Backend Engineer Evaluation

Before You Start — Read This

This assignment has three properties you should know about up front.

1. Parts of this spec are ambiguous or incomplete. That is intentional.

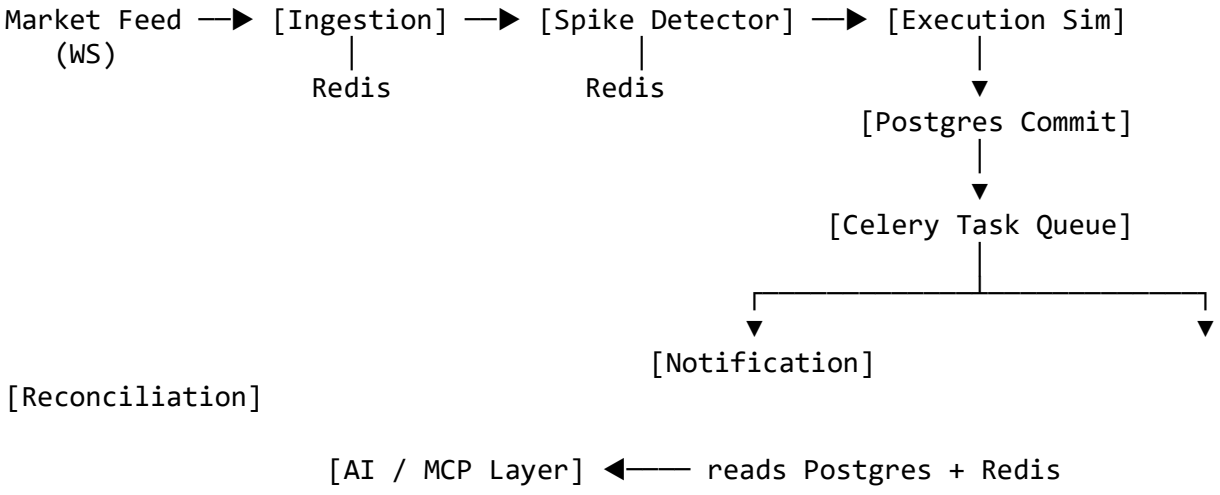
Ask. Asking costs nothing and is scored positively. Guessing silently is scored negatively. There is a questions channel; use it. An engineer who asks the right question on day one is worth more to us than one who builds the wrong thing beautifully.

2. Your engine will be tested against data you have not seen.

At submission, we will run your system against a replay file we provide. It contains real-world market feed conditions. Your engine must survive it. Build accordingly.

The System

Build a low-latency options execution engine that ingests a live market feed, detects rapid price movements, simulates option trades, persists them durably, dispatches notifications asynchronously, and exposes an AI-powered query layer over the results.



Part 1 — Real-Time Ingestion & Spike Detection

Stack

FastAPI, Redis, DhanHQ WebSocket.

WebSocket Consumer

Connect to the Dhan Feed using the `dhanhq` Python library. Subscribe to live LTP updates for:

- Instrument: NIFTY 50
- Security ID: `13`

Handle the connection lifecycle. The feed will disconnect. It will send you malformed frames. It will go silent during market close. Your process must not die, and it must not silently stop consuming.

The Replay Endpoint (Required)

`POST /debug/replay`

Accepts a newline-delimited JSON tick file. Each tick has the shape:

```
{"security_id": "13", "ltp": 22450.5, "ts": "2026-07-10T09:31:04.221Z"}
```

This endpoint must push ticks through the exact same pipeline as the live WebSocket. Not a parallel code path. Not a test harness that bypasses your detector. The same code.

This is how we will evaluate your engine, and how you should evaluate it yourself. Markets are closed most of the time.

Redis Window Management

Maintain a rolling 60-second price window in Redis. You may use Sorted Sets, Streams, or Lists — choose and justify in the README.

The window must efficiently:

- Append incoming ticks
- Expire ticks older than 60 seconds
- Retrieve the price at approximately `t - 60s` with minimal latency

Spike Detection Logic

On every incoming tick:

1. Read current price `Pt`
2. Fetch the price approximately 60 seconds ago, `P(t-60)`

Long Signal

If `(Pt - P(t-60)) / P(t-60) >= 0.05` → trigger Long → buy ATM Call.

Short Signal

If `(Pt - P(t-60)) / P(t-60) <= -0.05` → trigger Short → buy ATM Put.

Part 2 — Quant Logic

ATM Strike Calculation

Implement dynamic ATM strike selection following standard NIFTY option-chain increments.

Spot Price	ATM Strike
22432	22450
22424	22400
22425	?

The third row is not a typo. Decide, document your decision, and defend it.

Order Simulation

On signal:

- Calculate the ATM strike
- Determine CE or PE
- Simulate execution
- Fetch or mock the option premium

Persist: strike, option type, entry premium, timestamp, signal reason.

Part 3 — Persistence (PostgreSQL - ASYNC)

`trades` Table

Column	Type
`id`	UUID
`instrument`	VARCHAR
`strike`	INTEGER
`option_type`	VARCHAR
`side`	VARCHAR
`entry_price`	FLOAT
`pnl`	FLOAT

Column	Type
`signal_reason`	TEXT
`created_at`	TIMESTAMP

Requirements

- Async DB operations throughout
- Transaction safety
- Commit trades in the database

Part 4 — Celery: Asynchronous Work

This section is weighted heavily. Read it carefully.

4a. Notification Task

After a successful database commit, enqueue a Celery task that sends a WhatsApp or webhook notification.

Providers: Twilio, UltraMsg, Meta WhatsApp Cloud API, or a mock HTTP endpoint you stand up yourself. The provider does not matter. **The delivery semantics do.**

Payload:

```
{"message": "Trade Alert! Long NIFTY 22450 CE entered at 14:05. Reason: +5% Spike."}
```

Requirements:

- **Retry with exponential backoff** on transient failure
- **A maximum retry count**, after which the task must not silently vanish
- **Idempotency.** If the same trade is enqueued twice, the user receives one message. Explain your mechanism.

4b. Periodic Reconciliation

Configure a Celery Beat schedule that runs every 60 seconds and answers one question: **are there trades in Postgres that have no successful notification?**

Explain how would you handle this in case of failure of notification

4c. Failure Semantics

Answer these in your README. There is no single right answer; there is a right *reasoning process*.

1. Postgres commit succeeds, then the Celery broker is unreachable. What happens to that notification? What should?

2. A Celery worker picks up a notification task, sends the WhatsApp message successfully, then crashes before acknowledging the task. What happens on redelivery?
3. Your Celery worker pool is 4 processes. Two hundred spikes fire in ten seconds. What happens to tick ingestion? To latency? What did you do about it?

Question 2 is the interesting one.

4d. Broker & Backend

Choose your broker (Redis, RabbitMQ) and result backend. Justify both in one paragraph. State the durability tradeoff you accepted.

Part 5 — AI Trade Intelligence Layer

MCP Server

Expose MCP-compatible tools:

- `get_last_trade()`
- `get_open_positions()`
- `get_pnl_summary()`
- `get_spike_events()`
- `get_best_strike_accuracy()`
- `generate_trade_chart()`

Natural Language Endpoint

`POST /ask`

Should handle:

- "What was the last trade?"
- "Show today's losing trades."
- "Which strike performed best?"
- "Compare CE vs PE profitability."

Integration

OpenAI API, Ollama, LangChain, LlamaIndex — your choice.

Part 6 — Performance

Hard requirement: p99 tick-to-signal latency under 50ms, measured during replay.

"Tick-to-signal" means: from the moment a tick enters your ingestion path to the moment the spike detector emits a decision. Not including the Postgres write. Not including Celery.

- The measurement code must be in the repo.
- The measurement method must be described in the README.
- Report p50, p95, p99, and max.

If you miss the target, say so and explain where the time goes. **A candidate who reports a missed target with an accurate breakdown scores higher than one who reports a hit target with no methodology.**

Deliverables

Provide a GitHub repo link and the repo must be public for us to access it

Code

- FastAPI application
- Redis integration
- PostgreSQL models and migrations
- WebSocket consumer
- Replay endpoint
- Celery worker + Beat configuration
- MCP server and tools
- LLM integration

DevOps

- `docker-compose.yml` that brings the whole system up with one command
- `.env.example`
- API documentation

README — Weighted Equally With Code

Your README must contain:

1. **Architecture decisions.** Redis structure choice. Broker choice. Index choice. One paragraph each, with the tradeoff you accepted.
2. **The Celery failure semantics answers** (Part 4c, questions 1–3).
3. **Your latency numbers**, with methodology.
4. **What you cut and why.** If you ran out of time, say what you would build next and in what order.
5. **The questions you asked us, and what you did with the answers.**
6. **One thing you got wrong.** Something you believed at hour two that turned out to be false by hour six. If nothing surprised you, you did not look hard enough.
7. **Architecture Design/Flow (Mermaid or Any visual)**

Suggested Stack

FastAPI · Redis · PostgreSQL · SQLAlchemy or SQLModel · DhanHQ · Celery · OpenAI or Ollama · FastMCP · Docker